LangGraph: Graph-Based Orchestration for Stateful LLM Applications

and its Applications

CAQ413 Seminar

Seminar Report submitted in partial fulfillment for the degree
B.Tech in Computer Science and Engineering (Artificial Intelligence)

Submitted by

Nikhil Jimmy Thomas (Reg No: MUT23CA055)



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE
(Affiliated to APJ Abdul Kalam Technological University)

October 2026

Muthoot Institute of Technology and Science (Autonomous)
Kochi

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE

CERTIFICATE

This is to certify that the seminar report entitled “LangGraph: Graph-Based Orchestration for Stateful LLM Applications and its Applications” submitted by Nikhil Jimmy Thomas (Reg No: MUT23CA055) of Semester VII B.Tech Computer Science and Engineering (AI) is a bonafide record of the work done under my supervision during the academic year 2026–2027.

Guide	**Coordinators**	**Head of the Department**
Ms. Nimmi M K	Ms. Thasni K N & Ms. Vishnupriya Vijay	Dr. Cerene Mariam Abraham
Asst. Professor	Asst. Professors	Associate Professor
Dept. of AI & DS	Dept. of AI & DS	Dept. of AI & DS

ACKNOWLEDGEMENTS

Writing this report took a lot of reading and rewriting, and I had help at every step. I would like to thank the people who made it possible.

My first thanks go to my seminar guide, Asst. Prof. *Ms. Nimmi M K*. Her steady encouragement and patient feedback helped me turn a loose interest in AI agents into a focused study of LangGraph. I am also grateful to my seminar coordinators, Asst. Prof. *Ms. Thasni K N* and Asst. Prof. *Ms. Vishnupriya Vijay*, for reviewing my drafts carefully and pointing out what needed to change.

I thank Dr. Cerene Mariam Abraham, Head of the Department of Artificial Intelligence and Data Science, for her support and for keeping the department a good place to learn. I also thank Dr. Neelakantan P. C., Principal, for the academic framework that made this seminar possible.

I am grateful to the management of Muthoot Institute of Technology & Science for the facilities and resources provided, and to the faculty and staff of the Artificial Intelligence and Data Science Department.

Finally, I thank my classmates in S7 CSAI. Their questions during practice runs made the presentation better, and their company made the long evenings shorter. To everyone else who helped in ways big or small, thank you.

Nikhil Jimmy Thomas
(Reg. No: MUT23CA055)

Abstract

A single call to a large language model (LLM) can answer a question, but most useful work takes several steps. The program has to decide what to do next, try again when something fails, and remember what it has already done. Fixed prompt chains cannot loop, and fully autonomous agents are hard to predict or debug. LangGraph, an open-source library from LangChain, offers a middle route. The developer describes the application as a graph: each node is a step, each edge is a move to the next step, and all nodes share one state object. The model does its thinking inside the nodes, while the graph decides the order.

This seminar studies LangGraph through the paper by Duan and Wang, *Exploration of LLM Multi-Agent Application Implementation Based on LangGraph+CrewAI* (arXiv:2411.18241). The authors pair LangGraph, which controls the flow, with CrewAI, which groups role-based agents into a team. They illustrate the pairing with three cases: an email assistant, a code generation and review loop, and a system that audits and forwards work orders. The report explains LangGraph's core ideas, reviews six related papers, walks through the paper's architecture and workflow, and discusses its strengths, weaknesses and uses. One point stands out. The paper's results are case studies rather than measurements, so its claims of better efficiency still need to be tested.

Keywords: LangGraph, CrewAI, Large Language Models, Multi-Agent Systems, Agent Orchestration.

Contents

Certificate i

Acknowledgements ii

Abstract iii

List of Figures vi

List of Tables vii

List of Abbreviations viii

1 Introduction 1

2 Literature Survey 3
- 2.1 ReAct: Synergizing Reasoning and Acting in Language Models (Yao et al., 2023) . . . 3
- 2.2 Reflexion: Language Agents with Verbal Reinforcement Learning (Shinn et al., 2023) . . . 3
- 2.3 AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation (Wu et al., 2023) . . . 4
- 2.4 MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework (Hong et al., 2024) . . . 4
- 2.5 A Survey on Large Language Model based Autonomous Agents (Wang et al., 2023) . . . 4
- 2.6 Agent AI with LangGraph: A Modular Framework for Enhancing Machine Translation Using LLMs (Wang and Duan, 2024) . . . 4

3 Materials and Methods 6
- 3.1 LangGraph: Core Concepts . . . 6
- 3.2 CrewAI Framework . . . 7
- 3.3 Development Environment . . . 8
- 3.4 Evaluation Criteria . . . 8

4 Methodology 9
- 4.1 Design Rationale . . . 9
- 4.2 System Architecture . . . 9
- 4.3 Integrating LangGraph with CrewAI . . . 10
- 4.4 Workflow . . . 10
- 4.5 Worked Example: Automatic Email Handling . . . 11

5 Results and Performance **16**
5.1 Application Case 1: Code Generation with Code Review 16
5.2 Application Case 2: Automatic Work-Order Processing 17
5.3 Key Findings 17
5.4 Comparison with Other Approaches 18
5.5 Discussion 18

6 Advantages **19**

7 Disadvantages **20**

8 Applications **21**

9 Conclusion **22**

A Bibliography **23**

B Presentation Slides **25**

C Sample Reference Papers **27**

D Viva Questions and Answers **28**

E Similarity Index Report **30**

F Alignment with Sustainable Development Goals (SDG) **31**

List of Figures

3.1 A minimal LangGraph agent: nodes, a conditional edge (router) and a loop . 7

4.1 Two-layer architecture: LangGraph controls the flow and CrewAI does the work inside each node 10
4.2 Workflow of the LangGraph+CrewAI system 11
4.3 Email workflow graph built with LangGraph+CrewAI (redrawn from Duan and Wang [1], Fig. 1) 12
4.4 LangGraph+CrewAI workflow code (transcribed from [1], Fig. 2) 13
4.5 Agent definition in CrewAI (transcribed from [1], Fig. 3) 14
4.6 Task definition in CrewAI (transcribed from [1], Fig. 4) 15

5.1 Code generation and code check loop built with LangGraph+CrewAI (redrawn from [1], Fig. 5) 16
5.2 Work-order processing with a crew of agents and a vector store (redrawn from the description in [1]; the agents' labels are illustrative) 17

B.1 Seminar presentation slides (1–20) 26

C.1 Base paper: "Exploration of LLM Multi-Agent Application Implementation Based on LangGraph+CrewAI" (first page) [1] 27

List of Tables

2.1 Summary of the literature survey 5

3.1 Development environment used in the base paper 8

5.1 Qualitative comparison of LLM application approaches 18

F.1 Mapping of the seminar topic to UN Sustainable Development Goals 31

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
FAISS	Facebook AI Similarity Search
HITL	Human-in-the-Loop
JS / TS	JavaScript / TypeScript
LLM	Large Language Model
MAS	Multi-Agent System
ML	Machine Learning
PM	Product Manager
RAG	Retrieval-Augmented Generation
SDG	Sustainable Development Goal
SOP	Standard Operating Procedure

Chapter 1

Introduction

Large language models have changed quickly. A few years ago they mostly answered questions. Today they can plan, write code, look things up and call other programs. The trouble starts when you try to build something real with one. Take a support bot. It has to work out what the customer wants, send the request to the right place, fetch details from a database and check the reply before sending it. A code assistant has to write code, run it, read the error and try again, sometimes many times over.

Work like this does not run in a straight line. It **branches**: a billing question goes one way and a technical one goes another. It **loops**: try again until the tests pass. And it needs **memory**, because later steps depend on earlier results. The two usual ways of building LLM applications both struggle here. A *chain* runs a fixed list of steps in order. It is predictable, but it cannot go back and it keeps no shared state. A *free-running agent* lets the model decide everything, including which tool to call and when to stop. It is flexible, but hard to test, and one bad decision can throw the whole run off course.

LangGraph is built for the space between these two. It is an open-source library from LangChain, released in 2024 under the MIT licence, with versions for Python and JavaScript/TypeScript [2]. Its design borrows from graph-processing systems such as Google's Pregel [3]. The developer draws the application as a graph. Nodes are steps of work, edges are the moves between them, and every node reads from and writes to one shared state. The developer fixes which paths are allowed, and the model does the work inside each step. LangGraph works with many model providers and is used for single agents, teams of agents, retrieval pipelines and workflows that pause for a human to approve.

The base paper for this seminar, by Duan and Wang [1], asks what happens when LangGraph is combined with CrewAI [4], a framework that organises agents into role-based teams. In their design LangGraph handles the flow: which step runs next, when to branch, when to loop, and what goes into the state. CrewAI handles the teamwork: which agent takes which task and how the agents cooperate. The authors show the combination on an email assistant, a code generation and review loop, and a ticket-handling system.

Objectives of the Seminar

- To explain why multi-step LLM applications need explicit control, and where chains and free-running agents fall short.
- To describe LangGraph's main building blocks: state, nodes, edges, conditional edges and checkpoints.
- To review earlier work on LLM agents and multi-agent systems.
- To study the LangGraph+CrewAI architecture, workflow and case studies in the base paper.
- To weigh up the advantages, limitations and uses of graph-based agent orchestration.

The report is organised as follows. Chapter 2 reviews the literature and Chapter 3 introduces the tools. Chapter 4 explains the method used in the base paper and Chapter 5 discusses its results. Chapters 6 to 8 cover advantages, disadvantages and applications, and Chapter 9 concludes.

Chapter 2

Literature Survey

This chapter looks at six papers that shaped thinking about LLM agents. For each one I describe the method, what it showed, and the limitation that matters most when you want a multi-agent system you can actually control.

2.1 ReAct: Synergizing Reasoning and Acting in Language Models (Yao et al., 2023)

ReAct [5] lets the model switch back and forth between three kinds of output. A *thought* is the model reasoning about what to do. An *action* is a tool call, such as a Wikipedia search. An *observation* is what the tool returned. Because the model can look things up mid-way, it relies less on guesswork, and ReAct did better than reasoning-only or acting-only baselines on HotpotQA, ALFWorld and WebShop. Most tool-using agents today follow this pattern. Its weakness is structural. The whole run is one linear loop with no saved state, so an early mistake carries forward and can spoil everything after it.

2.2 Reflexion: Language Agents with Verbal Reinforcement Learning (Shinn et al., 2023)

Reflexion [6] lets an agent learn from failure without retraining the model. After a failed attempt the agent writes a short note in plain language about what went wrong. The note is kept in memory and given to the agent on its next try. This simple idea brought large gains on coding and decision-making tasks. The catch is that the agent needs a clear signal of whether it succeeded, which many real tasks do not provide. Every retry also costs more model calls and more time.

2.3 AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation (Wu et al., 2023)

AutoGen [7], an open-source framework from Microsoft, builds an application out of several agents that solve a problem by talking to each other. An agent can be backed by a model, by tools, by a person, or by a mix of these, so the framework is very flexible. The cost of that flexibility is control. The order of work comes out of the conversation itself, so it is hard to say in advance what the system will do, and agents can go in circles or wander off the task.

2.4 MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework (Hong et al., 2024)

MetaGPT [8] copies how a software company works. Standard operating procedures (SOPs) are written into the prompts. Agents take roles such as product manager, architect and engineer, and pass structured documents along an assembly line, checking each other's work. The paper showed that this structure cuts down the inconsistencies seen when agents simply chat. The pipeline is fixed, though, and built mainly for software projects, and running many agents uses a lot of tokens.

2.5 A Survey on Large Language Model based Autonomous Agents (Wang et al., 2023)

Wang et al. [9] surveyed the rapidly growing work on LLM-based agents. They proposed a common framework of four modules: *profile* (the agent's role), *memory*, *planning* and *action*. They also reviewed applications and evaluation methods. The survey is a useful map of the field. As a survey, however, it offers no implementation or orchestration tool, and the field moves faster than any survey can keep up with.

2.6 Agent AI with LangGraph: A Modular Framework for Enhancing Machine Translation Using LLMs (Wang and Duan, 2024)

In an earlier paper by the same two authors [10], LangGraph reads the input text and passes it to a specialised translation agent, with separate agents for English, French and Japanese built on models such as GPT-4o. It is a clear, concrete example of LangGraph's

routing. The evaluation is small, though: it covers only a few languages and depends on a paid model.

Summary

Put together, these papers show steady progress. ReAct gave agents the ability to act, Reflexion the ability to learn from mistakes, AutoGen the ability to converse, and MetaGPT the ability to follow procedures. They also show a gap that keeps coming back. Either the flow is rigid and cannot branch or loop, or it is left to the model and becomes hard to control. What is missing is a way for the developer to lay out the allowed paths, with shared state, loops and recovery, while the model still does the clever work at each step. LangGraph, and the LangGraph+CrewAI pairing in the base paper, aim at that gap.

Table 2.1: Summary of the literature survey

Paper	**Key idea**	**Main limitation**
ReAct [5]	Alternates reasoning with tool calls	Single linear loop; no saved state; errors carry forward
Reflexion [6]	Agent writes notes on its failures and retries	Needs a clear success signal; extra model calls
AutoGen [7]	Agents solve tasks by conversing	Flow comes from the conversation, so it is hard to control
MetaGPT [8]	Role-based agents follow SOPs	Fixed pipeline aimed at software projects
Agent survey [9]	Profile, memory, planning, action framework	No concrete orchestration tool
LangGraph for MT [10]	LangGraph routes text to translation agents	Few languages; small evaluation

Chapter 3

Materials and Methods

The base paper does not train a model or use a dataset. Its materials are the frameworks and tools the system is built from. This chapter introduces them, along with the criteria the paper uses to judge the result.

3.1 LangGraph: Core Concepts

LangGraph builds stateful, multi-step LLM applications as graphs [2]. The base paper highlights three things it adds over other LLM frameworks: loops, fine-grained control over the flow, and memory that persists between steps [1]. It supports several shapes of workflow, including single-agent, multi-agent, hierarchical and sequential. Its building blocks are listed below and drawn in Fig. 3.1.

- **State:** the shared data, usually a typed dictionary, that every node reads and updates. A reducer can say how an update is merged; for example, a new message can be appended to a list instead of replacing it.
- **Node:** an ordinary Python function that does one step of work, such as calling the model or a tool, and returns the part of the state it changed.
- **Edge:** a fixed move: after node A, run node B.
- **Conditional edge:** a small router function that looks at the state and picks the next node. Branches and loops are built from these.
- **START and END:** the graph's entry and exit points.
- **Checkpointer:** saves the state after every step. A run can then be paused, inspected, resumed after a crash, or continued once a person has approved it.

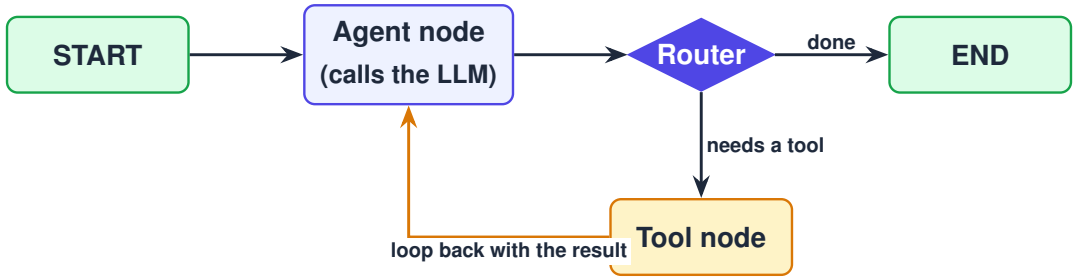


Figure 3.1: A minimal LangGraph agent: nodes, a conditional edge (router) and a loop

A LangGraph application is built in five steps: define the state, add nodes as functions, connect them with edges, compile the graph, and run it. The short example below answers a question with a single model call.

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class State(TypedDict):          # 1. define the state
    question: str
    answer: str

def answer(state: State):        # 2. a node is a plain function
    return {"answer": llm.invoke(state["question"])}

g = StateGraph(State)
g.add_node("answer", answer)
g.add_edge(START, "answer")      # 3. connect the edges
g.add_edge("answer", END)
app = g.compile()                # 4. compile
app.invoke({"question": "What is LangGraph?"})   # 5. run
```

3.2 CrewAI Framework

CrewAI [4] is an open-source framework for coordinating role-playing agents that work on a problem together. Each agent is given a role, a goal and a set of tools. The framework has four building blocks:

- **Agent:** a unit that can carry out tasks, make decisions and talk to other agents. It is described by a role, a goal and a short backstory.
- **Task:** a piece of work for an agent, with a description, the agent responsible and the tools it may use.
- **Tool:** something an agent can use, such as reading email or searching documents.

- **Crew:** a team of agents, plus the process that decides the order in which their tasks run.

CrewAI can use model APIs such as OpenAI's as well as locally run models through Ollama [11]. It supports agents with custom roles, automatic hand-off of tasks between agents, and flexible task management [4]. It is designed for multi-step work, decision-making and problem-solving.

3.3 Development Environment

Table 3.1 lists the components the base paper uses to build its cases [1].

Table 3.1: Development environment used in the base paper

Component	**Tool**	**Purpose**
Multi-agent framework	CrewAI [4]	Organises and coordinates the agents' work
Graph workflow	LangGraph [2]	Shares state between nodes and controls the flow
Workflow tracking	LangSmith [12]	Monitors and audits each run
Embedding model	Ollama embeddings [11]	Turns work-order text into vectors for retrieval
Vector database	FAISS [13]	Stores the vectors and finds similar ones quickly

3.4 Evaluation Criteria

The base paper is an exploratory case study. It reports no accuracy, speed or cost figures. Instead it judges the LangGraph+CrewAI pairing qualitatively, on four points that I also use in this report:

- **Control:** can the developer define and restrict the flow of the application?
- **Information flow:** how easily do state and results pass between steps and agents?
- **Collaboration:** how well is the work split among agents with different roles?
- **Extensibility:** how easily can existing tools and agents, such as those built with LangChain, be plugged in?

Chapter 4

Methodology

4.1 Design Rationale

The base paper starts from a simple observation. A single LLM agent cannot handle complex tasks well, and a team of agents only works if two things are in place: tight control of the flow and good teamwork [1]. Instead of asking one framework to do both, the authors give each job to the tool that suits it. LangGraph's graph structure provides the control and moves information between agents. CrewAI's role-based crews decide who does what. The paper asks two questions: how to design agents on LangGraph so that the flow is precisely controlled, and how CrewAI can make those agents capable of a wider range of tasks.

4.2 System Architecture

The architecture has two layers that meet at the shared state (Fig. 4.1).

- **LangGraph, the control layer.** The user's task enters the graph. Nodes stand for agent steps, and edges with conditional routing decide which step runs next. The shared state carries data between steps.
- **CrewAI, the team layer.** Inside a node, a crew of agents does the actual work. Each agent has a role and a goal, tasks are assigned to agents, and the agents work together to produce the node's result.

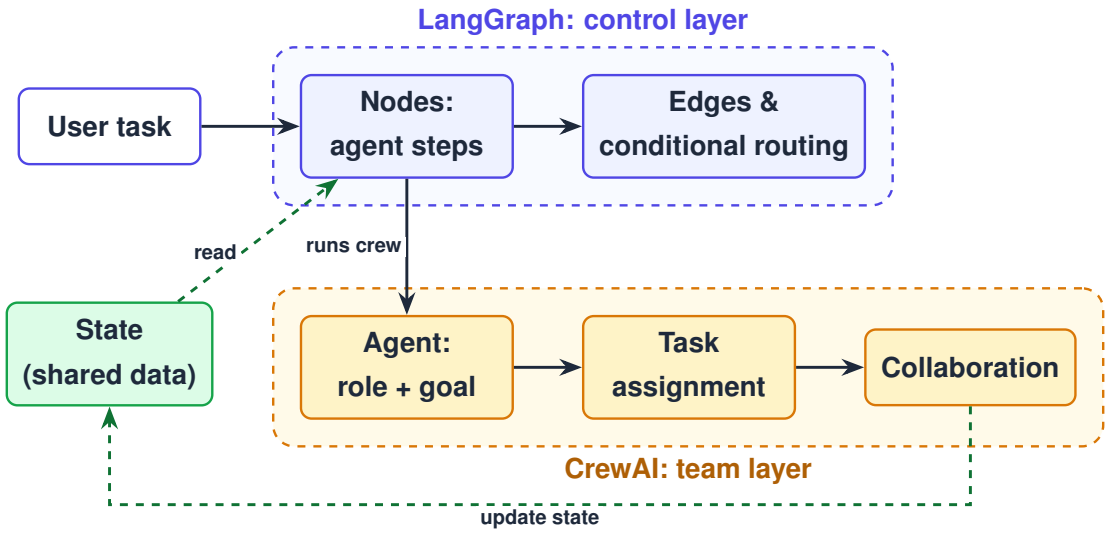


Figure 4.1: Two-layer architecture: LangGraph controls the flow and CrewAI does the work inside each node

4.3 Integrating LangGraph with CrewAI

According to the authors, the combined framework makes complex tasks easier to manage. Flexible workflows, cooperation between agents and graph-structured management together improve how tasks are carried out, and the framework can be connected to existing tools. CrewAI keeps the work organised by assigning tasks clearly and defining roles. Because CrewAI also works with LangChain, developers who already have LangChain agents can bring them in without rewriting them [1].

In practice the split is easy to state:

- **LangGraph decides the flow:** the order of steps, the branches, the loops and the state.
- **CrewAI decides the teamwork:** which role-based agent takes which task, and how the agents cooperate.

4.4 Workflow

Fig. 4.2 follows a task through the combined system. LangGraph receives the task and routes it to the next node. The CrewAI crew in that node carries out the work, and the result is written back to the shared state. A conditional edge then checks whether the job is finished. If not, the flow moves to the next step; if it is, the graph ends.

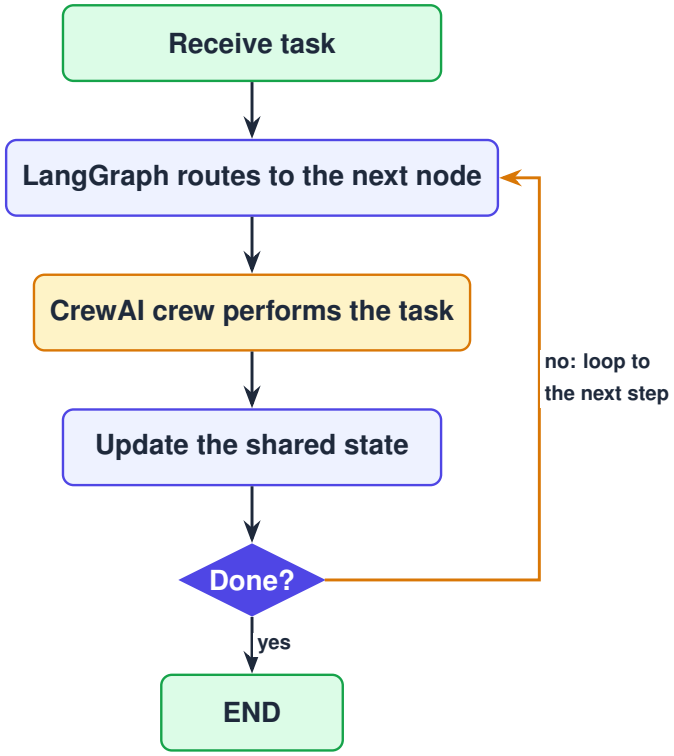


Figure 4.2: Workflow of the LangGraph+CrewAI system

4.5 Worked Example: Automatic Email Handling

To make the method concrete, the authors take the email example from CrewAI's documentation and rebuild it on LangGraph+CrewAI [1]. The job is split into small steps and run automatically.

Fig. 4.3 shows the graph. It starts at `check_new_emails`. A conditional edge then either continues to `draft_responses`, if new mail has arrived, or goes straight to `wait_next_run`. After waiting, the graph returns to check the inbox again. That loop is what turns a one-off script into an assistant that keeps running, and it is something a plain chain cannot express.

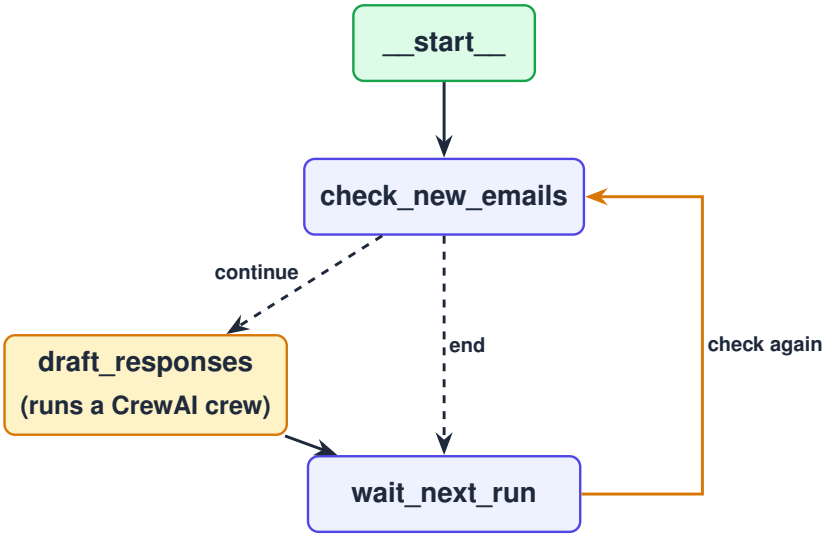


Figure 4.3: Email workflow graph built with LangGraph+CrewAI (redrawn from Duan and Wang [1], Fig. 1)

The code that builds this graph (Fig. 4.4) follows the five steps from Chapter 3. The most interesting line is the `draft_responses` node. Its function is `EmailFilterCrew().kickoff`, so a whole CrewAI crew runs as one LangGraph node.

```python
from dotenv import load_dotenv
load_dotenv()
from langgraph.graph import StateGraph
from .state import EmailsState
from .nodes import Nodes
from .crew.crew import EmailFilterCrew

class WorkFlow():
    def __init__(self):
        nodes = Nodes()
        workflow = StateGraph(EmailsState)
        workflow.add_node("check_new_emails", nodes.check_email)
        workflow.add_node("wait_next_run", nodes.wait_next_run)
        workflow.add_node("draft_responses", EmailFilterCrew().kickoff)
        workflow.set_entry_point("check_new_emails")
        workflow.add_conditional_edges(
            "check_new_emails",
            nodes.new_emails,
            {"continue": 'draft_responses', "end": 'wait_next_run'})
        workflow.add_edge('draft_responses', 'wait_next_run')
        workflow.add_edge('wait_next_run', 'check_new_emails')
        self.app = workflow.compile()
```

Figure 4.4: LangGraph+CrewAI workflow code (transcribed from [1], Fig. 2)

Inside the crew, each agent is defined by a role, a goal and a backstory (Fig. 4.5). The “Senior Email Analyst”, for example, filters out newsletters and promotions. A task (Fig. 4.6) then spells out what the agent has to do, what its answer should look like, and which agent is responsible.

```
class EmailFilterAgents():
    def __init__(self):
        self.gmail = GmailToolkit()

    def email_filter_agent(self):
        return Agent(
            role='Senior Email Analyst',
            goal='Filter out non-essential emails like newsletters and promotional
    content',
            backstory=dedent("""\
                As a Senior Email Analyst, you have extensive experience in email
    content analysis.
                You are adept at distinguishing important emails from spam,
    newsletters, and other
                irrelevant content. Your expertise lies in identifying key patterns
    and markers that
                signify the importance of an email."""),
            verbose=True,
            allow_delegation=False
        )
```

Figure 4.5: Agent definition in CrewAI (transcribed from [1], Fig. 3)

```
class EmailFilterTasks:
    def filter_emails_task(self, agent, emails):
        return Task(
            description=dedent(f"""\
                Analyze a batch of emails and filter out
                non-essential ones such as newsletters, promotional content and
    notifications.

                Use your expertise in email content analysis to distinguish
                important emails from the rest, pay attention to the sender and
    avoid invalid emails.

                Make sure to filter for the messages actually directed at the user
    and avoid notifications.

                EMAILS
                -------
                {emails}

                Your final answer MUST be the relevant thread_ids and the sender,
    use bullet points.
                """),
            agent=agent
        )
```

Figure 4.6: Task definition in CrewAI (transcribed from [1], Fig. 4)

Chapter 5

Results and Performance

The base paper presents its results as worked cases. It shows that the LangGraph+CrewAI design can be used to build working multi-agent applications and describes what the pairing improves. This chapter goes through those cases, lists the main findings, and then looks at what the results do and do not prove.

5.1 Application Case 1: Code Generation with Code Review

The authors used LangGraph+CrewAI to build an application that writes code and then reviews it (Fig. 5.1). A question goes to a *code generation* node, which writes the code. A *code check* node then tests it. If the check fails, the flow goes back to generation along with the feedback. If it passes, the code is returned as the answer. Because the status of each attempt and the feedback are shared through the graph state as they happen, the authors report that code generation became more efficient [1]. This is the same write, test and fix cycle that Reflexion and ReAct argue for. The difference is that here the loop is a visible edge in the graph rather than something left to the model.

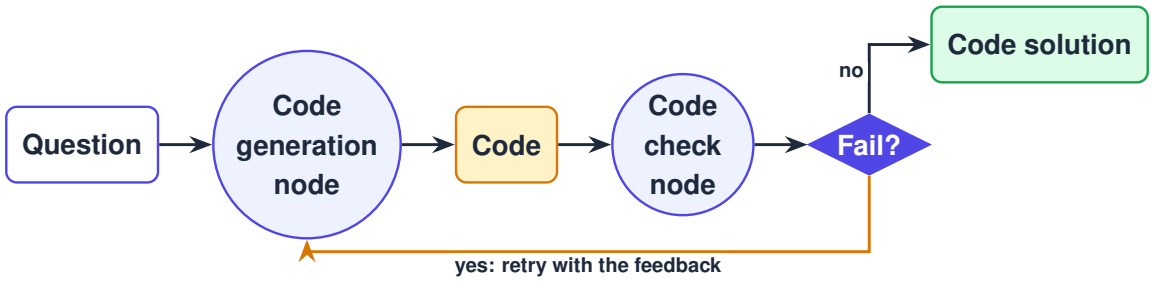


Figure 5.1: Code generation and code check loop built with LangGraph+CrewAI (redrawn from [1], Fig. 5)

5.2 Application Case 2: Automatic Work-Order Processing

The second case checks incoming work orders (tickets) and forwards them (Fig. 5.2). The text of each ticket is turned into a vector with an Ollama embedding model and stored in FAISS, so that similar past tickets can be found. Several CrewAI agents, each with its own task, then study the ticket in detail. By sharing what they find, the agents understand tickets more accurately, and the authors report faster ticket handling as a result [1].

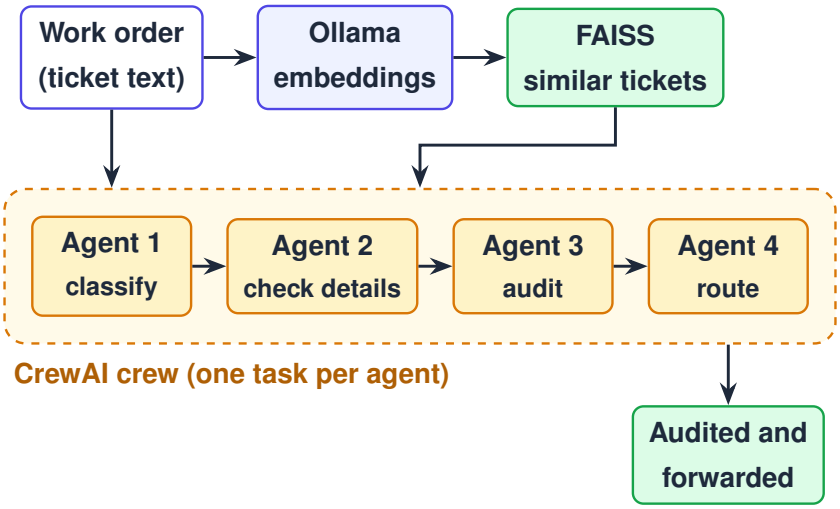


Figure 5.2: Work-order processing with a crew of agents and a vector store (redrawn from the description in [1]; the agents' labels are illustrative)

5.3 Key Findings

From these cases the authors draw four conclusions:

- **Efficient information flow:** LangGraph's graph structure makes it easier to pass information between agents.
- **Precise control:** building agents on LangGraph gives the developer close control over the multi-agent workflow.
- **Better collaboration:** CrewAI's task allocation and resource management improve teamwork and overall performance.
- **Complex tasks:** by splitting the work and cooperating, the agents handle tasks that a single agent finds hard.

5.4 Comparison with Other Approaches

Table 5.1 compares the approaches discussed in this report on the properties that matter when you want a multi-agent system you can control. The comparison is qualitative and based on the papers reviewed in Chapter 2.

Table 5.1: Qualitative comparison of LLM application approaches

Approach	**Flow set by**	**Loops**	**Saved state**	**Human-in-the-loop**	**Role-based teams**	**Predictable**
Simple chain	Developer	No	No	No	No	High
ReAct agent	LLM	Yes	No	Limited	No	Low
AutoGen	Conversation	Yes	Partial	Yes	Yes	Low
MetaGPT	Fixed SOP	Limited	Documents	No	Yes	Medium
LangGraph alone	Developer (graph)	Yes	Yes (checkpoints)	Yes	Manual	High
LangGraph + CrewAI	**Developer (graph)**	**Yes**	**Yes**	**Yes**	**Yes (crews)**	**High**

5.5 Discussion

The base paper does a good job of showing *how* LangGraph and CrewAI fit together. Its three cases make the division of work easy to grasp: the graph decides what happens next, and the crew decides who does it. The examples are practical, too. Sorting email, reviewing code and handling tickets are everyday jobs in many organisations.

The results need to be read carefully, though. The paper reports no measurements at all: no success rates, no response times, no costs, and no comparison with a single agent or with another framework. When the authors say efficiency “improved”, they are describing what they saw in their cases, not the outcome of an experiment. A useful next step would be to test the pairing on a benchmark, measuring task success, the number of model calls and the cost against a single agent and against LangGraph on its own. Even with that gap, the paper is a clear and readable guide to building multi-agent applications in which control and teamwork sit in separate, well-defined layers.

Chapter 6

Advantages

- **Explicit control:** the developer decides which paths the application can take. The model fills in each step but cannot invent a new route, so the behaviour is easier to predict and to audit.
- **Shared state:** every step sees the same, up-to-date data, so nothing has to be copied between agents or squeezed into prompts.
- **Loops and retries:** a step can repeat until a condition is met, for example regenerating code until the tests pass. A plain chain cannot do this.
- **Persistence:** because the state is saved after every step, an application can pause, resume and recover from a failure without starting again [2].
- **Human-in-the-loop:** the graph can stop at any node and wait for a person to check or edit the state, which matters for actions such as sending an email.
- **Easy to debug:** each node is a normal function that can be tested on its own, and tracing tools such as LangSmith [12] show exactly which path a run took.
- **Works with many models:** both LangGraph and CrewAI can use several providers, including local models through Ollama.
- **Clear teamwork with CrewAI:** roles, goals and tasks give each agent a defined job, and existing LangChain agents can join a crew with little extra work.

Chapter 7

Disadvantages

- **Learning curve:** graphs, typed state, reducers and checkpointers take time to learn, and using two frameworks means learning two sets of ideas.
- **More code for simple jobs:** a task that a basic chain handles in a few lines needs more setup as a graph.
- **Large graphs get messy:** with many nodes and conditional routes, the flow becomes hard to picture and reason about.
- **Cost and delay:** loops and multiple agents mean more model calls, which raises both the cost and the response time.
- **Still depends on the model:** the graph limits the routes, but a poor model output can still pick the wrong route or produce a weak result inside a node.
- **Fast-changing tools:** both frameworks change their APIs between versions, so code and tutorials go out of date quickly.
- **No numbers in the base paper:** the benefits are shown through cases only, so they are not yet backed by measured results.

Chapter 8

Applications

- **Customer support:** a router node classifies each query and sends it to a billing agent, a technical agent or a person, keeping the conversation in the shared state.
- **RAG question answering:** the graph fetches documents, scores how relevant they are and, if they are poor, rewrites the query and searches again before answering.
- **Code generation and bug fixing:** write, test and fix in a loop, as in the base paper's code review case and in the authors' later study of automated bug fixing with LangGraph [14].
- **Machine translation:** text is routed to language-specific translation agents [10].
- **Data analysis:** separate agents build and run machine learning pipelines on Apache Spark [15].
- **Multi-agent teams:** crews for research, writing and review, with one role per agent, as proposed in the base paper [1].
- **Ticket and workflow automation:** checking, classifying and forwarding work orders, with similar past cases found in a vector database.

Chapter 9

Conclusion

LangGraph turns an LLM application into a clear graph of steps that share one state. It adds what simple chains lack, namely branches, loops, memory and human control, while avoiding the unpredictability of fully autonomous agents. The developer decides which paths are allowed, and the model does the thinking within each step.

The base paper by Duan and Wang shows what happens when this idea is combined with CrewAI. LangGraph manages the flow and the state, CrewAI organises role-based agents into teams, and together they take on tasks that a single agent struggles with. The email assistant, the code review loop and the ticket system make the approach concrete and show that it fits ordinary, everyday problems.

The paper's main weakness is its evaluation. Its benefits are shown through cases rather than experiments, so the efficiency gains it describes still need to be confirmed with proper benchmarks of success rate, cost and response time. Future work could also look at how such systems behave as the number of agents grows, and how to keep large graphs easy to understand.

Overall, graph-based orchestration is fast becoming a common way to build dependable agentic AI. Studying it closely for this seminar showed me why: when the structure is explicit, the model's mistakes become much easier to find, contain and fix.

Appendix A

Bibliography

[1] Z. Duan and J. Wang, “Exploration of LLM multi-agent application implementation based on LangGraph+CrewAI,” *arXiv preprint arXiv:2411.18241*, 2024.

[2] LangChain Inc., “LangGraph documentation,” https://langchain-ai.github.io/langgraph/, 2025.

[3] G. Malewicz, M. H. Austern, A. J. C. Bik, J. C. Dehnert, I. Horn, N. Leiser, and G. Czajkowski, “Pregel: A system for large-scale graph processing,” in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2010, pp. 135–146.

[4] CrewAI Inc., “CrewAI documentation,” https://docs.crewai.com, 2025.

[5] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023, arXiv:2210.03629.

[6] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, arXiv:2303.11366.

[7] Q. Wu, G. Bansal, J. Zhang, Y. Wu *et al.*, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” *arXiv preprint arXiv:2308.08155*, 2023.

[8] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng *et al.*, “MetaGPT: Meta programming for a multi-agent collaborative framework,” in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2308.00352.

[9] L. Wang, C. Ma, X. Feng, Z. Zhang *et al.*, “A survey on large language model based autonomous agents,” *arXiv preprint arXiv:2308.11432*, 2023.

[10] J. Wang and Z. Duan, “Agent AI with LangGraph: A modular framework for enhancing machine translation using large language models,” *arXiv preprint arXiv:2412.03801*, 2024.

[11] Ollama, “Ollama documentation,” https://github.com/ollama/ollama, 2025.

[12] LangChain Inc., “LangSmith documentation,” https://docs.smith.langchain.com, 2025.

[13] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.

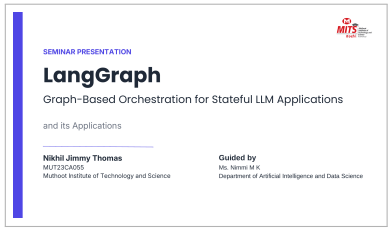
[14] J. Wang and Z. Duan, “Empirical research on utilizing LLM-based agents for automated bug fixing via LangGraph,” *arXiv preprint arXiv:2502.18465*, 2025.

[15] ——, “Intelligent spark agents: A modular LangGraph framework for scalable, visualized, and enhanced big data machine learning workflows,” *arXiv preprint arXiv:2412.01490*, 2024.

Appendix B

Presentation Slides

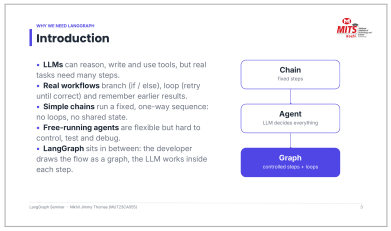
The seminar was presented with the 20 slides shown on the next page.



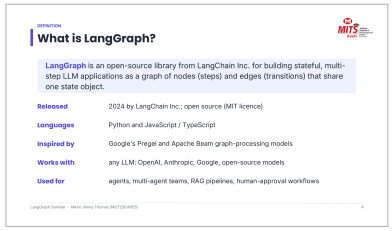
Slide 01



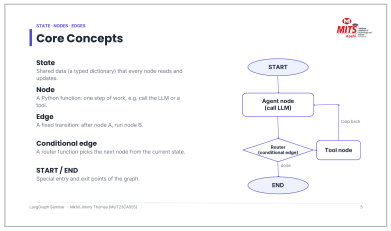
Slide 02



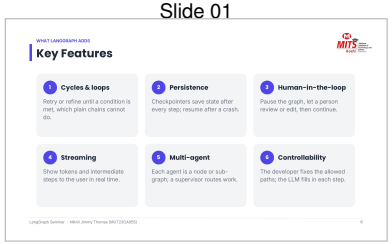
Slide 03



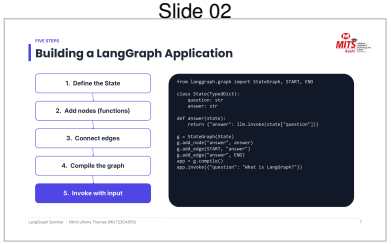
Slide 04



Slide 05



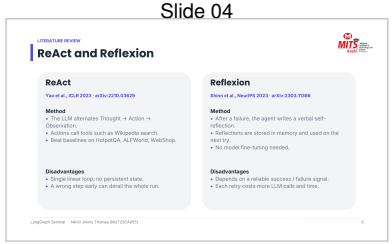
Slide 06



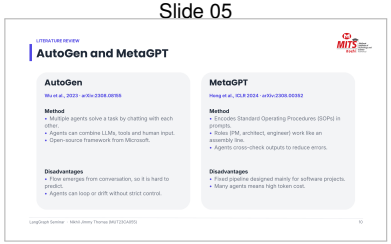
Slide 07



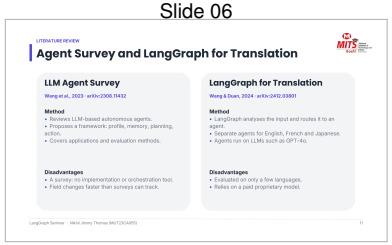
Slide 08



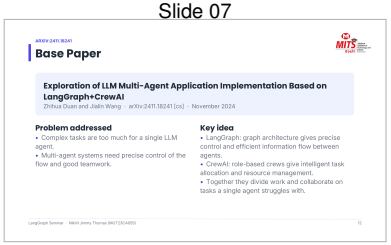
Slide 09



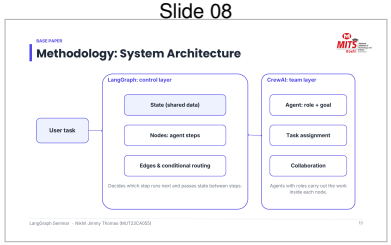
Slide 10



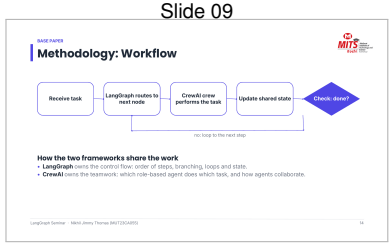
Slide 11



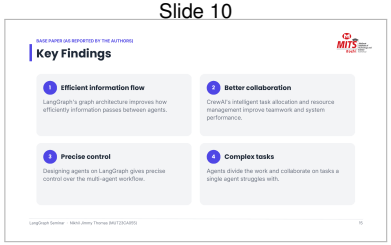
Slide 12



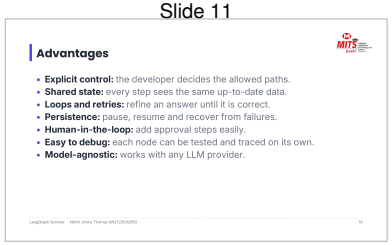
Slide 13



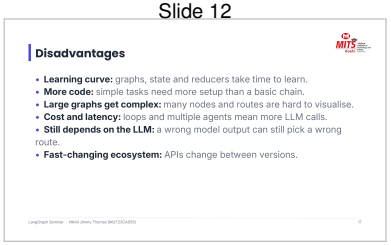
Slide 14



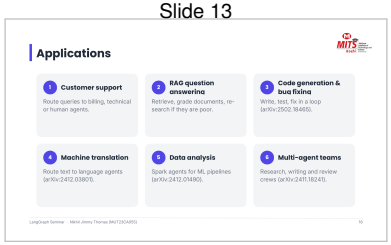
Slide 15



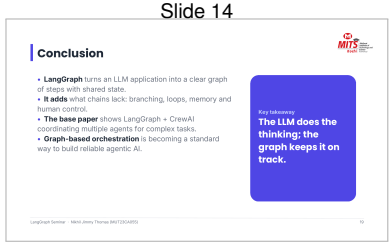
Slide 16



Slide 17



Slide 18



Slide 19

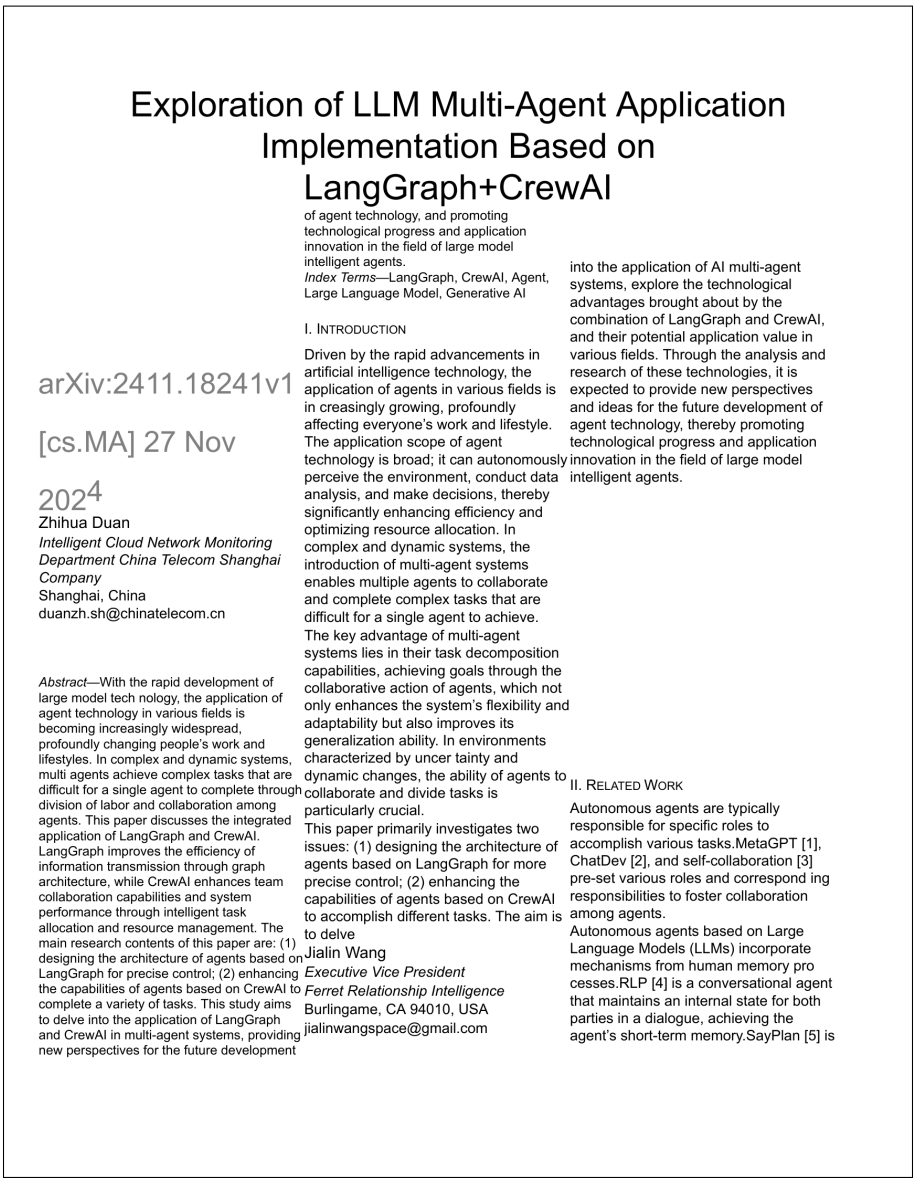


Slide 20

Figure B.1: Seminar presentation slides (1–20)

Appendix C

Sample Reference Papers



Exploration of LLM Multi-Agent Application Implementation Based on LangGraph+CrewAI

arXiv:2411.18241v1 [cs.MA] 27 Nov 2024

Zhihua Duan
Intelligent Cloud Network Monitoring Department China Telecom Shanghai Company
Shanghai, China
duanzh.sh@chinatelecom.cn

Abstract—With the rapid development of large model tech nology, the application of agent technology in various fields is becoming increasingly widespread, profoundly changing people's work and lifestyles. In complex and dynamic systems, multi agents achieve complex tasks that are difficult for a single agent to complete through division of labor and collaboration among agents. This paper discusses the integrated application of LangGraph and CrewAI. LangGraph improves the efficiency of information transmission through graph architecture, while CrewAI enhances team collaboration capabilities and system performance through intelligent task allocation and resource management. The main research contents of this paper are: (1) designing the architecture of agents based on LangGraph for precise control; (2) enhancing the capabilities of agents based on CrewAI to complete a variety of tasks. This study aims to delve into the application of LangGraph and CrewAI in multi-agent systems, providing new perspectives for the future development of agent technology, and promoting technological progress and application innovation in the field of large model intelligent agents.

Index Terms—LangGraph, CrewAI, Agent, Large Language Model, Generative AI

I. Introduction

Driven by the rapid advancements in artificial intelligence technology, the application of agents in various fields is in creasingly growing, profoundly affecting everyone's work and lifestyle. The application scope of agent technology is broad; it can autonomously perceive the environment, conduct data analysis, and make decisions, thereby significantly enhancing efficiency and optimizing resource allocation. In complex and dynamic systems, the introduction of multi-agent systems enables multiple agents to collaborate and complete complex tasks that are difficult for a single agent to achieve. The key advantage of multi-agent systems lies in their task decomposition capabilities, achieving goals through the collaborative action of agents, which not only enhances the system's flexibility and adaptability but also improves its generalization ability. In environments characterized by uncer tainty and dynamic changes, the ability of agents to collaborate and divide tasks is particularly crucial.

This paper primarily investigates two issues: (1) designing the architecture of agents based on LangGraph for more precise control; (2) enhancing the capabilities of agents based on CrewAI to accomplish different tasks. The aim is to delve into the application of AI multi-agent systems, explore the technological advantages brought about by the combination of LangGraph and CrewAI, and their potential application value in various fields. Through the analysis and research of these technologies, it is expected to provide new perspectives and ideas for the future development of agent technology, thereby promoting technological progress and application innovation in the field of large model intelligent agents.

Jialin Wang
Executive Vice President
Ferret Relationship Intelligence
Burlingame, CA 94010, USA
jialinwangspace@gmail.com

II. Related Work

Autonomous agents are typically responsible for specific roles to accomplish various tasks.MetaGPT [1], ChatDev [2], and self-collaboration [3] pre-set various roles and correspond ing responsibilities to foster collaboration among agents.

Autonomous agents based on Large Language Models (LLMs) incorporate mechanisms from human memory pro cesses.RLP [4] is a conversational agent that maintains an internal state for both parties in a dialogue, achieving the agent's short-term memory.SayPlan [5] is

Figure C.1: Base paper: "Exploration of LLM Multi-Agent Application Implementation Based on LangGraph+CrewAI" (first page) [1]

Appendix D

Viva Questions and Answers

Common Viva Questions

1. **What is LangGraph?**
   Answer: An open-source library from LangChain for building stateful, multi-step LLM applications as a graph. Nodes are steps, edges are the moves between them, and all nodes share one state object.

2. **How is LangGraph different from a LangChain chain?**
   Answer: A chain runs a fixed sequence of steps in one direction. A LangGraph graph can branch with conditional edges, loop back to earlier steps, keep shared state and save checkpoints, so it can retry, pause and resume.

3. **What are state, nodes and edges?**
   Answer: The state is shared data, usually a typed dictionary, that every node reads and updates. A node is a function that does one step. An edge is a move between nodes; a conditional edge uses a router function to pick the next node from the current state.

4. **What is a checkpointer and why is it useful?**
   Answer: It saves the graph state after every step. A run can then resume after a crash, be inspected for debugging, or wait for a person's approval before continuing.

5. **What does CrewAI add to LangGraph in the base paper?**
   Answer: Role-based teamwork. Agents are given roles, goals and tools, and tasks are shared among them. LangGraph controls the order of steps and the state, while a CrewAI crew does the work inside a node.

6. **Explain the email example from the base paper.**
   Answer: The graph starts at `check_new_emails`. If there is new mail it continues to `draft_responses`, which runs a CrewAI crew; otherwise it goes to `wait_next_run`. After waiting, it loops back to check again.

7. **What tools did the authors use besides LangGraph and CrewAI?**
   Answer: LangSmith to track and audit workflows, an Ollama embedding model to turn work-order text into vectors, and FAISS as the vector database for fast retrieval.

8. **How does LangGraph compare with AutoGen?**

Answer: In AutoGen the order of work comes out of the agents' conversation, which is flexible but hard to predict. In LangGraph the developer lays out the allowed paths, so the behaviour is easier to control and debug.

9. **What are the limitations of the base paper?**
   Answer: Its results are case studies. It gives no measurements of accuracy, cost or speed and no comparison with a baseline, so the reported efficiency gains are not yet backed by experiments.

10. **Where can LangGraph be applied?**
    Answer: Customer support routing, RAG question answering, code generation and bug fixing, machine translation, data analysis pipelines, multi-agent research and writing teams, and ticket automation.

Appendix E

Similarity Index Report

[Insert Similarity Index Report Here]

Appendix F

Alignment with Sustainable Development Goals (SDG)

Table F.1: Mapping of the seminar topic to UN Sustainable Development Goals

SDG	**Rationale for alignment**
SDG 9: Industry, Innovation and Infrastructure (primary)	Open-source orchestration frameworks such as LangGraph make it easier and cheaper to build reliable AI systems, supporting innovation in industry and research.
SDG 4: Quality Education	Graph-based agents can power tutoring and question-answering assistants, and LangGraph's explicit structure makes agentic AI easier for students to learn.